Tipos de índices en Oracle

Bases de Datos 2 · Ingeniería de Sistemas · Sesión de dos horas

La pregunta de hoy

- Todos sabemos crear un índice. La pregunta es cuál, sobre qué columnas, y sobre todo cuándo no crearlo.
- Oracle ofrece una docena de tipos. Cada uno resuelve un problema distinto y ninguno es gratis.

Las dos horas

- 0:00 Qué es un índice y qué cuesta tenerlo
- 0:15 B-tree: único, compuesto, de cobertura · Demos 1 y 2
- 0:45 Bitmap y bitmap join · Demo 3
- 1:10 Funcional, descendente, clave inversa, comprimido, IOT · Demo 4
- 1:35 Particionados y gestión: invisible, unusable, auditoría · Demo 5
- 1:50 Lo que cuesta un índice · Demo 6, y cierre

El trato que se hace al crear un índice

Lo que gana

- Encontrar unas pocas filas sin leer la tabla entera. En una tabla grande, la diferencia es de miles de bloques a unas decenas.
- Y, según el tipo, también: ordenar sin ordenar, contar sin contar y hacer cumplir la unicidad.

Lo que paga, siempre

- Espacio en disco, y con él más respaldo y más tiempo de recuperación.
- Mantenimiento en cada INSERT, en cada DELETE y en cada UPDATE de esa columna. La carga de datos se vuelve más lenta.
- Un plan más que el optimizador debe considerar, y una oportunidad más de que elija mal.

La consecuencia práctica

- **Un índice se justifica con una consulta concreta que lo necesite. Sin esa consulta, es solo costo.**
- **Y el que ya no se usa hay que borrarlo: sigue cobrando aunque nadie lo lea.**

El mapa completo

POR ESTRUCTURA
B-tree (el que sale por defecto)
Bitmap
Tabla organizada por índice
Cluster y hash cluster

POR CONTENIDO
Sobre columnas
Basado en función
Sobre columna virtual
Bitmap join
Texto (Oracle Text)

POR PROPIEDADES
Único / no único
Ascendente / descendente
Clave inversa
Comprimido

POR PARTICIONAMIENTO
No particionado
Local (una pieza por partición)
Global (una sola estructura)

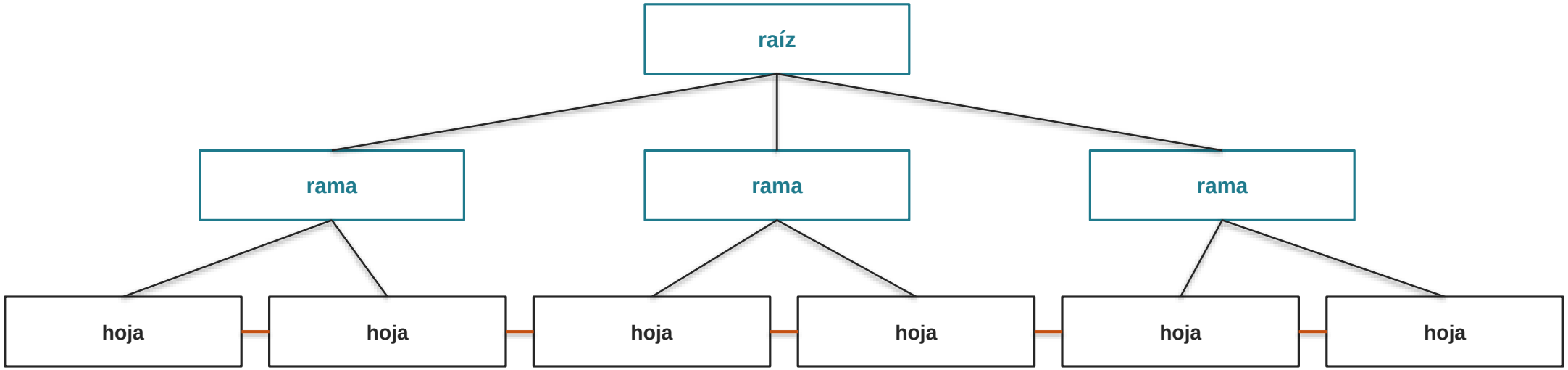
POR VISIBILIDAD
Visible / invisible
Usable / unusable
Válido / en construcción

LOS QUE VEREMOS HOY
B-tree, bitmap, funcional,
inverso, comprimido, IOT,
local y global

El 90 % de los índices de cualquier base son B-tree sobre columnas. Los demás resuelven casos específicos

Bitmap y particionamiento requieren Enterprise Edition o XE: en Standard Edition 2 no existen

B-tree · la estructura por defecto



Las hojas guardan el valor ordenado y el rowid de su fila, y están encadenadas entre sí

Altura constante	todas las hojas están a la misma distancia de la raíz: buscar cuesta lo mismo esté donde esté el valor
Dos o tres saltos	basta para millones de filas; por eso escala tan bien
Hojas encadenadas	de ahí que sirva para rangos, BETWEEN y ORDER BY, no solo para igualdad
No guarda nulos	una fila con la columna en NULL no entra al índice

B-tree · las variantes que hay que dominar

Único y no único

- El único, además de acelerar, impide duplicados. Es el que Oracle crea solo al declarar PRIMARY KEY o UNIQUE.
- **Si lo que quiere es la regla, declare la restricción y deje que Oracle cree el índice: queda documentada en el esquema.**

Compuesto o concatenado

- Sobre varias columnas. Sirve para el prefijo izquierdo: (A, B, C) responde por A, por A y B, y por las tres.
- No responde por B solo. La excepción de Oracle es el INDEX SKIP SCAN, y solo cuando A tiene muy pocos valores distintos.
- **La primera columna debe ser aquella por la que siempre se filtra, no la más selectiva.**

De cobertura

- Cuando el índice contiene todas las columnas que la consulta pide, el motor no toca la tabla. Desaparece el TABLE ACCESS BY INDEX ROWID.
- **Demos 1 y 2 — 01_btree.sql**

Bitmap · cómo se guarda

los datos **los mapas de bits que guarda el índice**

fila	canal	estado	WEB	TIENDA	PAGADA	PEND.	ANUL.
1	WEB	PAGADA	1	0	1	0	0
2	TIENDA	PAGADA	0	1	1	0	0
3	WEB	PENDIENTE	1	0	0	1	0
4	TIENDA	ANULADA	0	1	0	0	1
5	WEB	PAGADA	1	0	1	0	0

Un mapa de bits por cada valor distinto. Un 1 dice que esa fila tiene ese valor.

WHERE canal = 'WEB' AND estado = 'PAGADA' → el motor hace un AND binario entre dos mapas: 10101 AND 11001 = 10001

Sin leer una sola fila de la tabla, ya sabe que las filas 1 y 5 cumplen. Un B-tree no puede hacer eso.

Y como es un mapa comprimido, ocupa una fracción de lo que ocuparía un B-tree sobre la misma columna

Bitmap · cuándo sí y cuándo jamás

Cuándo sí

- Columnas de baja cardinalidad: sexo, estado, canal, región, año. Pocos valores distintos sobre muchas filas.
- Consultas que combinan varios filtros poco selectivos: es el patrón de cualquier tablero analítico.
- Bodegas de datos y tablas que se cargan por lotes y luego solo se consultan.
- Conteos: un COUNT(*) con filtros se resuelve sumando bits, sin tocar la tabla.

Cuándo jamás

- **OLTP con escritura concurrente. Un bitmap bloquea el fragmento del mapa, no la fila: al actualizar una fila quedan bloqueadas todas las que comparten ese valor.**
- Columnas de alta cardinalidad: un mapa por valor distinto sobre un millón de valores es un desastre.

Bitmap join index

- Indexa una columna de otra tabla y resuelve el join por adelantado. Potentísimo en bodegas, y frágil: si cambia la dimensión, hay que reconstruirlo.
- **Demo 3 — 02_bitmap.sql, incluido el bloqueo con dos sesiones**

B-tree contra bitmap, lado a lado

	B-TREE	BITMAP
Cardinalidad ideal	alta: muchos valores distintos	baja: pocos valores distintos
Estructura	árbol balanceado con rowids	mapa de bits por valor
Tamaño	grande	muy compacto
Combinar filtros	uno por consulta, en general	AND y OR entre varios mapas
Nulos	no los indexa	sí los indexa
DML concurrente	bloquea por fila	bloquea por fragmento del mapa
Entorno	OLTP y también analítico	solo bodega de datos
Edición	todas	Enterprise y XE

La pregunta que decide: ¿esta tabla se escribe todo el día? Si la respuesta es sí, no hay bitmap que valga

Índice basado en función

El problema que resuelve

- Un índice guarda el valor tal como está en la tabla. Si el WHERE le aplica una función, el valor buscado no existe en el árbol y el índice queda inservible.

```
WHERE UPPER(nombre) = 'GARCIA'          -- escaneo completo
WHERE TRUNC(fecha_venta) = DATE '...'   -- escaneo completo
```

La solución

```
CREATE INDEX fx_cliente_upper ON cliente (UPPER(nombre));
```

- Se indexa la expresión. El predicado tiene que escribirse EXACTAMENTE igual que la expresión indexada, o no se usa.

Dos advertencias y un truco

- La función debe ser determinística: para la misma entrada, siempre la misma salida. SYSDATE no sirve.
- **Antes de crearlo, pregúntese si no es mejor arreglar la consulta. Un rango de fechas evita el índice funcional por completo.**
- **El truco: un CASE que devuelva NULL para las filas que no interesan produce un índice parcial, porque los nulos no se indexan.**

Descendente y de clave inversa

Descendente

```
CREATE INDEX ix ON venta (id_sucursal ASC, fecha_venta DESC);
```

- Guarda una columna en orden inverso. Sobre una sola columna no aporta nada: un B-tree ya se recorre en los dos sentidos.
- Sirve cuando el ORDER BY mezcla direcciones. Curiosidad: Oracle lo implementa internamente como índice basado en función.

Clave inversa

```
CREATE UNIQUE INDEX rx ON auditoria (id_evento) REVERSE;
```

- Invierte los bytes de la clave. Su razón de ser es un problema de concurrencia: cuando la clave viene de una secuencia, todas las sesiones insertan en el mismo bloque hoja y ahí se forma la fila.
- Al invertir, valores consecutivos caen en bloques distintos y la escritura se reparte. Es una técnica clásica en RAC.
- **El precio es alto: al perder el orden, los rangos dejan de funcionar. Sirve solo para búsquedas por igualdad.**
- ▸ **Demo 4 — 03_especiales.sql**

Comprimido y tabla organizada por índice

Índice comprimido

```
CREATE INDEX ix ON venta (canal, estado, id_cliente)
  COMPRESS 2;
```

Los prefijos repetidos se guardan una sola vez por bloque hoja.

Menos bloques, menos lecturas, menos memoria.

Cuesta algo de CPU al leer.

Solo vale la pena si las primeras columnas se repiten mucho.

Tabla organizada por índice

```
CREATE TABLE ticket (...)
  ORGANIZATION INDEX;
```

La tabla ES el índice: las filas viven ordenadas dentro del B-tree de la clave primaria. No hay segundo salto a la tabla.

Ideal en tablas de cruce y de detalle.

Mala idea si la tabla es ancha o tiene muchos índices secundarios.



Heap: dos accesos

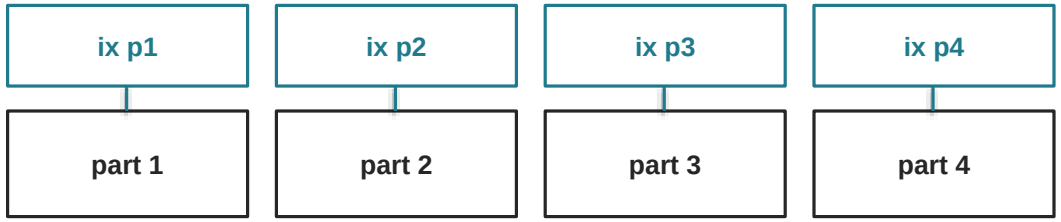


IOT: uno solo

Las dos son optimizaciones de segundo orden: primero acierte el B-tree, y solo después piense en estas

Índices sobre tablas particionadas

LOCAL

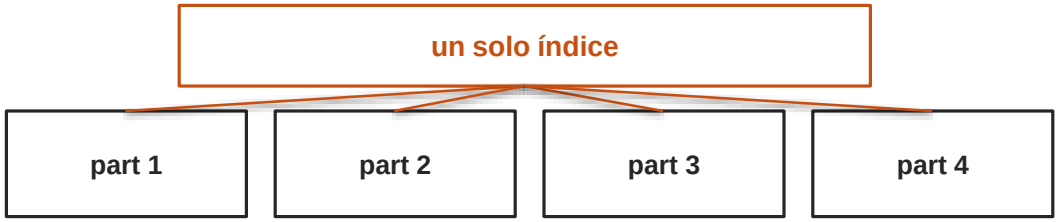


Una pieza de índice por partición de la tabla.

Al borrar o archivar una partición, solo desaparece su pieza.

Permite recortar particiones (pruning) cuando el filtro trae la clave de partición.

GLOBAL



Una sola estructura que cubre toda la tabla.

Al tocar una partición, el índice entero queda inutilizable

salvo que se use UPDATE GLOBAL INDEXES, que cuesta tiempo.

Regla práctica: LOCAL por defecto en tablas particionadas por fecha con purga o archivado

GLOBAL solo cuando el acceso nunca lleva la clave de particionamiento, o cuando hace falta unicidad en toda la tabla

Demo 5 — 04_particionados_y_gestion.sql

Invisible, inutilizable y monitoreado

Invisible

```
ALTER INDEX ix_sospechoso INVISIBLE;
```

- Sigue existiendo y se mantiene con el DML, pero el optimizador lo ignora. Es la forma segura de averiguar si un índice hace falta antes de borrarlo.
- **También sirve al revés: crear un índice invisible en producción y activarlo cuando se haya probado.**

Inutilizable

```
ALTER INDEX ix UNUSABLE;   ...carga masiva...   ALTER INDEX ix REBUILD;
```

- Se apaga el índice para no pagar el mantenimiento fila a fila durante una carga grande, y se reconstruye de una vez al final.
- **Cuidado: si el índice sostiene una PRIMARY KEY o una UNIQUE, dejarlo inutilizable bloquea el DML de la tabla.**

¿Alguien lo usa?

```
SELECT name, used, total_access_count, last_used FROM v$index_usage_info;
```

- Desde 12.2 Oracle lleva la cuenta solo. En versiones anteriores había que activar MONITORING USAGE índice por índice.

Cuándo NO crear el índice

Por la consulta

- Cuando el filtro devuelve buena parte de la tabla: leerla de corrido sale más barato que ir y volver miles de veces.
- Cuando el predicado le aplica una función a la columna: arregle primero la consulta.
- Cuando el LIKE empieza con comodín. Ahí no hay B-tree que sirva; se necesita Oracle Text.

Por la columna

- Baja cardinalidad en una tabla OLTP: el B-tree no descarta nada y el bitmap no se puede usar.
- Columnas que se actualizan constantemente: cada UPDATE paga el mantenimiento.

Por lo que ya existe

- Si ya hay un índice cuyo prefijo izquierdo sirve, el nuevo sobra. Dos índices sobre (A) y (A, B) suelen ser uno de más.
- **Y en tablas pequeñas, las que caben en pocos bloques, el escaneo completo casi siempre gana.**

Cómo decidir, en cinco preguntas

1. **¿Hay una consulta lenta concreta que lo pida?**
   Si no la hay, no hay índice. Punto.

2. **¿Qué porcentaje de la tabla devuelve el filtro?**
   Por encima de un umbral, el escaneo completo gana

3. **¿La columna tiene muchos valores distintos?**
   Muchos → B-tree. Pocos → bitmap, y solo si no hay DML concurrente

4. **¿El predicado toca la columna tal cual?**
   Si le aplica una función, arregle la consulta antes de indexar la expresión

5. **¿Existe ya un índice con ese prefijo izquierdo?**
   Si existe, no cree otro: amplíelo o déjelo como está

Y una sexta, la que más se olvida: cree el índice, vuelva a medir, y bórrelo si no mejoró nada

Lo que cuesta un índice, medido

En espacio

- En una tabla muy indexada, los índices llegan a pesar tanto como los datos. Ese espacio se respalda y se restaura también.

En escritura

- Cada índice se mantiene en cada INSERT y DELETE, y en los UPDATE de sus columnas. Cinco índices pueden multiplicar por varias veces el tiempo de una carga.
- **Es la respuesta concreta a la pregunta de siempre: ¿y por qué no indexamos todas las columnas?**

En estabilidad

- Cada índice es un plan más que el optimizador puede elegir. Más opciones significa más probabilidad de que un día elija mal.

El demo

- Cargamos las mismas 200.000 filas en dos tablas idénticas, una sin índices y otra con cinco, y comparamos el reloj. La diferencia se ve sin necesidad de explicarla.
- ▸ **Demo 6 — 05_costo_del_indice.sql**

Lo que hay que llevarse

Seis frases

- El B-tree resuelve el 90 % de los casos. Empiece siempre por ahí.
- El bitmap es para bodegas de datos. En OLTP genera contención, no rendimiento.
- El índice funcional es el último recurso: primero intente reescribir el predicado.
- La clave inversa arregla un problema de concurrencia y rompe las búsquedas por rango.
- En tablas particionadas, LOCAL por defecto.
- **Un índice que nadie usa sigue cobrando. Hágalo invisible, espere, y bórrelo.**

Autoevaluación

- ¿Por qué un bitmap sobre una columna de alta cardinalidad es mala idea?
- ¿Qué gana y qué pierde un índice de clave inversa?
- ¿Cuándo un índice de cobertura deja de servir?
- ¿Por qué conviene LOCAL en una tabla particionada por fecha con purga mensual?
- Tiene (A) y (A, B) sobre la misma tabla. ¿Cuál borra y por qué?